



Using FFmpeg with NVIDIA GPU Hardware Acceleration

Release 13.1

NVIDIA Corporation

May 22, 2026

Contents

1	Introduction	1
2	FFmpeg Feature Matrix	3
2.1	Codec Support by FFmpeg Version	3
3	Setup	5
3.1	Hardware Setup	5
3.2	Software Setup	5
3.2.1	Using Pre-compiled FFmpeg	6
3.2.2	Compiling FFmpeg	6
3.2.2.1	Compiling for Linux	6
3.2.2.2	Compiling for Windows	7
3.2.2.3	Commonly faced issues and tips to resolve them	8
4	Basic Testing	9
4.1	1:1 Simple Transcoding	9
4.2	1:1 Transcode with Scaling	10
4.3	1:1 Transcode with Pixel Format Conversion	10
4.4	1:N Transcode with Scaling	11
4.5	1:N Encode from YUV or RAW Data	11
4.6	Multiple 1:N Transcode with Scaling	11
4.7	Multiple 1:N Transcode with Scaling (SW Decode->HW Scaling->HW Encode)	12
5	Video Decoding	15
6	Video Encoding	17
6.1	Understanding Key NVENC Parameters	17
6.1.1	Presets	17
6.1.2	Tuning Info	17
6.1.3	Rate Control Modes	18
6.1.4	B-frames and Reference Frames	18
6.1.5	GOP (Group of Pictures) Structure	19
6.1.6	Bitrate Control	19
6.1.7	Lookahead	20
6.1.8	Adaptive Quantization (AQ)	20
6.1.9	Temporal Filtering	21
6.1.10	Multi-Pass Encoding	21
6.1.11	Split Frame Encoding	22
6.2	Use Case Recommendations	22
6.2.1	Recording and Archiving	23
6.2.2	Game and Studio Broadcasting	23
6.2.3	Low-Latency Streaming and Real-Time Applications	24
6.2.4	Lossless Encoding	26
6.2.5	Memory-Constrained Encoding	26

6.2.6	High-Quality Live Streaming	27
6.2.7	Multi-Stream Adaptive Bitrate (ABR) Ladder	27
6.2.8	Video Editing Workflows	28
6.2.9	High-Resolution Encoding (8K)	29
7	Performance Evaluation and Optimization	31
7.1	Measuring Aggregate Performance	31
7.2	Monitoring GPU and Codec Utilization with nvidia-smi dmon	31
7.3	Decoder Performance: Keep Frames on the GPU	32
7.4	Settings for Reduced Initialization Time	32

Chapter 1. Introduction

Fully-accelerated hardware video encoding and decoding is supported on NVIDIA® GPUs of the Turing generation or newer. The hardware encoder and hardware decoder are referred to as NVENC and NVDEC, respectively, throughout this document.

The hardware capabilities of NVENC and NVDEC are exposed in the [NVIDIA Video Codec SDK](#) through APIs (referred to as NVENC API and NVDEC API), which provide access to the hardware acceleration features of NVENC and NVDEC.

FFmpeg is the most popular multimedia transcoding software and is used extensively for video and audio transcoding. By leveraging NVENC and NVDEC with FFmpeg, you can significantly accelerate video decoding, encoding, and end-to-end transcoding workflows.

This document explains how to use FFmpeg with NVIDIA GPU hardware acceleration through the APIs exposed in the NVIDIA Video Codec SDK.

Note

FFmpeg with HW acceleration is supported on the Jetson Thor platform (including Jetson T4000 and T5000 modules) with JetPack 7.1 and later.

Chapter 2. FFmpeg Feature Matrix

The following tables summarize NVENC (encoding) and decoding (NVDEC and CUVID) capabilities across major FFmpeg versions. Both decoding interfaces use the same NVDEC hardware.

2.1. Codec Support by FFmpeg Version

Table 1: NVENC Features by FFmpeg Version

FFmpeg Version	Major NVENC Features
2.6	Initial NVENC with support for H.264 4:2:0 8-bit
2.7	NVENC with support for HEVC 4:2:0 8-bit
2.8	2-pass encoding, YV12 support, H.264 4:4:4 8-bit
3.0	Better DTS generation, B-frame fixes
3.1	CUDA frames support, lossless encoding
3.2	Lookahead, HEVC 10-bit , HEVC 4:4:4
3.3	Per-frame QP control, Bluray compatibility, dynamic CUDA loading
3.4	Weighted prediction, fractional CQ
4.0	D3D11 surface input, H.264 B-frames as reference, P016 format
4.1	A53 closed captions, dynamic bitrate/aspect ratio changes
4.2	HEVC B-frames as reference
4.3	CQ mode improvements, multiple reference frames
4.4	HEVC timecode, more SEI data support
5.0	Intra refresh, chroma QP offsets, user data unregistered SEI
5.1	Additional 10-bit RGB formats
6.0	AV1 4:2:0 8-bit and 10-bit
6.1	Slice size limiting, RGB subsampling control
7.0	SDK 12.2 bit depth API

continues on next page

Table 1 – continued from previous page

FFmpeg Version	Major NVENC Features
7.1	Split-frame encoding, unidirectional B-frames, UHQ tune, HEVC temporal filtering
8.0	MV-HEVC, HEVC/H.264 4:2:2 8-bit/10-bit, H.264 4:2:0 10-bit

Table 2: NVDEC/CUVID Features by FFmpeg Version

FFmpeg Version	Major NVDEC/CUVID Features
3.1	Initial CUVID decoder with H.264 4:2:0 8-bit, HEVC 4:2:0 8-bit
3.2	Deinterlacing, HEVC 4:2:0 10-bit, VP8/VP9/VC-1/MPEG-1/MPEG-2/MPEG-4 4:2:0 8-bit
3.3	Cropping/resizing, P010/P016 support, dynamic CUDA loading
3.4	Capability checks, hw_device_ctx API
4.0	NVDEC hwaccels (H.264, HEVC, VC-1, MPEG-1/2/4, VP8, VP9), HEVC 4:2:0 12-bit, MJPEG 4:2:0 8-bit
4.1	CUDA yadif deinterlacer, frame pool optimizations
4.2	HEVC 4:4:4 8-bit/10-bit/12-bit
4.3	Support for aarch64
4.4	AV1 4:2:0 8-bit/10-bit , film grain
5.0	Decoder optimization
5.1	AV1 fixes
6.0	Frame copy improvements, buffer management
6.1	RefStruct API, dynamic surface allocation, MPEG fixes
7.0	RefStruct-pool API, bitstream management
7.1	Pixel format improvements
8.0	H.264 4:2:0 10-bit, H.264 4:2:2 8-bit/10-bit, HEVC 4:2:2 8-bit/10-bit/12-bit

Note

CUVID is both a parser and a decoder; NVDEC is a decoding-only layer. See the [Video Decoding](#) section for how the two approaches differ, supported codecs, and when to use each.

Chapter 3. Setup

3.1. Hardware Setup

FFmpeg with NVIDIA GPU acceleration requires a system with a supported NVIDIA GPU and either Linux, Windows, or Windows Subsystem for Linux (WSL) operating system.

Requirements:

- ▶ A supported NVIDIA GPU with NVENC and NVDEC capabilities (Turing generation or newer)
- ▶ Appropriate NVIDIA GPU driver installed for your operating system
- ▶ For a complete list of supported GPUs and their codec capabilities, refer to the [Video Encode and Decode Support Matrix](#).

Throughout this document, it is assumed that your system has a GPU with both NVENC and NVDEC capabilities and that the appropriate NVIDIA drivers are installed.

Tip

To verify your GPU and driver installation, use `nvidia-smi` command in your terminal or command prompt.

3.2. Software Setup

Note

Before using FFmpeg, it is recommended to refer to the FFmpeg documentation, note the version of the Video Codec SDK it uses, and ensure that the minimum GPU driver required for that version of the Video Codec SDK is installed.

CUDA is available through the NVIDIA driver API. The CUDA runtime library is not required to run the examples in this document. With NPP deprecated in FFmpeg (see the Compiling FFmpeg section), all of those examples use only the driver API.

3.2.1. Using Pre-compiled FFmpeg

If you prefer not to compile FFmpeg from source, pre-compiled binaries are available for download from the [official FFmpeg download page](#).

Available options:

- **Linux:** Static and shared builds, and distribution-specific packages (Ubuntu, Debian, Fedora, etc.)
- **Windows:** Pre-built executables from various trusted sources

Note

When using pre-compiled FFmpeg binaries, ensure they are built with NVENC/NVDEC support enabled. You can verify this by running `ffmpeg -encoders | grep nvenc` and `ffmpeg -decoders | grep cuvid`.

For users who need custom compilation options or want the latest features, follow the instructions in the next section to compile FFmpeg from source.

3.2.2. Compiling FFmpeg

FFmpeg is an open-source project. Download the FFmpeg source code repository and compile it using an appropriate compiler.

More information on building FFmpeg can be found at: [FFmpeg Compilation Guide](#).

Note

CUDA NPP has been deprecated in FFmpeg for CUDA versions above 12.8. It is recommended to avoid `--enable-libnpp` when compiling FFmpeg.

3.2.2.1 Compiling for Linux

FFmpeg with NVIDIA GPU acceleration is supported on Linux, including Windows Subsystem for Linux (WSL) and Jetson Thor (JetPack 7.1 and later; T4000 and T5000 modules).

To compile FFmpeg on Linux, do the following:

- Install necessary packages using your distribution's package manager. For Ubuntu/Debian:

```
sudo apt update && sudo apt -y install git build-essential pkg-config nasm  
→ clang
```

- Clone and install the NVIDIA codec headers (ffnvcodec):

```
git clone https://code.ffmpeg.org/FFmpeg/nv-codec-headers.git  
cd nv-codec-headers && sudo make install && cd -
```

- Clone FFmpeg's public Git repository:

```
git clone https://code.ffmpeg.org/FFmpeg/FFmpeg.git
```

- Configure FFmpeg with NVIDIA GPU support:

```
cd FFmpeg && ./configure --prefix="$PWD/build" --enable-nonfree --disable-
static --enable-shared --nvcc=clang
```

- Compile FFmpeg (adjust -j value based on your CPU cores, or use -j\$(nproc) for automatic detection):

```
make -j$(nproc)
```

- Install FFmpeg to the local build directory:

```
make install
```

- Set the library path to use the locally-built FFmpeg (add this to your ~/.bashrc for persistence):

```
export LD_LIBRARY_PATH=$PWD/build/lib:$LD_LIBRARY_PATH
export PATH=$PWD/build/bin:$PATH
```

- (Optional) Verify the build by checking for NVENC/NVDEC support:

```
ffmpeg -hide_banner -encoders | grep nvenc
ffmpeg -hide_banner -decoders | grep cuvid
ffmpeg -hide_banner -filters | grep cuda
```

Note

On Jetson Thor, the NVIDIA Video Codec SDK (NVENC/NVDEC) was introduced with JetPack 7.1. For more information, see the [NVIDIA Developer Blog announcement](#).

3.2.2.2 Compiling for Windows

FFmpeg with NVIDIA GPU acceleration is supported on all Windows platforms, with compilation through MSYS2.

To compile FFmpeg on Windows, do the following:

- Download and install MSYS2 from <https://www.msys2.org/> with default settings (installs to C:\msys64). Open “MSYS2 MSYS” from Start Menu and update:

```
pacman -Syu
```

- Launch UCRT64 environment (C:\msys64\ucrt64.exe or “MSYS2 UCRT64” in Start Menu) and install build tools:

```
pacman -S --needed base-devel git \
  mingw-w64-ucrt-x86_64-toolchain \
  mingw-w64-ucrt-x86_64-clang \
  mingw-w64-ucrt-x86_64-pkg-config \
  mingw-w64-ucrt-x86_64-nasm
```

- Clone and install the NVIDIA codec headers (ffnvcodec):

```
mkdir -p ~/ffmpeg-work/deps && cd ~/ffmpeg-work/deps
git clone https://code.ffmpeg.org/FFmpeg/nv-codec-headers.git
cd nv-codec-headers && make install PREFIX=/ucrt64 && cd -
```

- Clone FFmpeg's public Git repository:

```
cd ~/ffmpeg-work
git clone https://code.ffmpeg.org/FFmpeg/FFmpeg.git ffmpeg-source
cd ffmpeg-source
```

- Configure FFmpeg with NVIDIA GPU support:

```
./configure \
--prefix="$PWD/build" \
--arch=x86_64 \
--target-os=mingw32 \
--enable-nonfree \
--disable-debug \
--disable-shared \
--enable-static \
--pkg-config-flags="--static" \
--extra-ldflags="-static -static-libgcc -static-libstdc++" \
--nvcc=clang
```

- Compile FFmpeg (adjust -j value based on your CPU cores, or use -j\$(nproc) for automatic detection):

```
make -j$(nproc)
```

- Install FFmpeg to the local build directory:

```
make install
```

- (Optional) Verify the build by checking for NVENC/NVDEC support:

```
./build/bin/ffmpeg.exe -hide_banner -encoders | grep nvenc
./build/bin/ffmpeg.exe -hide_banner -decoders | grep cuvid
./build/bin/ffmpeg.exe -hide_banner -filters | grep cuda
```

The compiled FFmpeg binary can be found in build/bin/ directory. You can copy the binaries to a location in your PATH for system-wide access.

3.2.2.3 Commonly faced issues and tips to resolve them

Common compilation issues:

- FFmpeg's master branch may be broken at times. Please check out a release version if it is broken, or use an older snapshot.

Common run-time issues:

- Use -fps_mode passthrough when encoding to preserve the original frame timing from the decoded source and prevent FFmpeg from duplicating or dropping frames due to frame rate conversion.

Chapter 4. Basic Testing

Once the FFmpeg binary with NVIDIA hardware acceleration support is compiled, hardware-accelerated video transcode should be tested to ensure everything works well.

Note

The code snippets in this section keep video frames in GPU memory using either NVDEC hwaccel (`-hwaccel cuda -hwaccel_output_format cuda`) or CUVID decoders (`-c:v <codec>_cuvid`). See the [Video Decoding](#) section for the distinction and supported codecs.

The NVENC plugin in FFmpeg supports the following codecs:

- ▶ `h264_nvenc` - H.264 Encoder
- ▶ `hevc_nvenc` - HEVC Encoder
- ▶ `av1_nvenc` - AV1 Encoder

The command lines in this document use `h264_nvenc`, which should be replaced by `hevc_nvenc` for HEVC encoding or `av1_nvenc` for AV1 encoding.

Tip

The notation **1:N** in subsection titles refers to transcoding workflows where **1** input is transcoded into **N** outputs (e.g., 1:2 means one input produces two outputs at different resolutions/bitrates).

Tip

For systems with multiple GPUs, use the `-hwaccel_device` option to specify which GPU to use for hardware acceleration. For example, `-hwaccel cuda -hwaccel_device 1` will use the second GPU (device numbering starts at 0). Alternatively, set the `CUDA_VISIBLE_DEVICES` environment variable to control which GPUs are visible to FFmpeg.

To list available GPUs and their device IDs, use `nvidia-smi -L`.

4.1. 1:1 Simple Transcoding

The following command reads file *input.mp4* and transcodes it to *output.mp4* with H.264 video at the same resolution and with the same audio codec.

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -c:a copy -
↳c:v h264_nvenc -b:v 5M -fps_mode passthrough output.mp4
```

4.2. 1:1 Transcode with Scaling

The following command reads file *input.mp4* and transcodes it to *output.mp4* with H.264 video at 720p resolution and with the same audio codec. This command uses the built-in resizer in the CUVID decoder.

```
ffmpeg -y -c:v h264_cuvid -resize 1280x720 -i input.mp4 -c:a copy -c:v h264_
↳nvenc -b:v 5M -fps_mode passthrough output.mp4
```

There is a built-in cropper in the CUVID decoder as well. The following command illustrates the use of cropping (-crop (top)x(bottom)x(left)x(right)):

```
ffmpeg -y -c:v h264_cuvid -crop 16x16x32x32 -i input.mp4 -c:a copy -c:v h264_
↳nvenc -b:v 5M -fps_mode passthrough output.mp4
```

Alternatively, the `scale_cuda` resize filter can be used as shown below:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -vf scale_
↳cuda=1280:720:interp_algo=bilinear -c:a copy -c:v h264_nvenc -b:v 5M -fps_
↳mode passthrough output.mp4
```

4.3. 1:1 Transcode with Pixel Format Conversion

The following command demonstrates transcoding with pixel format conversion, a common workflow in broadcast environments where contribution feeds (4:2:2 10-bit) need to be converted to distribution formats (4:2:0 10-bit). This command reads a P210 (4:2:2 10-bit) input and transcodes it to P010 (4:2:0 10-bit) output, performing the chroma subsampling conversion on the GPU.

Input: *input_P210.mp4* (4:2:2 10-bit)

Output: *output_P010.mp4* (4:2:0 10-bit)

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input_P210.mp4 -vf
↳scale_cuda=format=p010 -c:a copy -c:v h264_nvenc -b:v 5M -fps_mode
↳passthrough output_P010.mp4
```

Note

This feature is particularly useful with GPUs that support 4:2:2 encoding and decoding, such as the Blackwell generation GPUs. The `scale_cuda` filter can convert between various pixel formats while keeping frames in GPU memory.

4.4. 1:N Transcode with Scaling

The following command reads file *input.mp4* and transcodes it to two different H.264 videos at various output resolutions and bit rates. Note that while using the GPU video encoder and decoder, this command also uses the scaling filter (*scale_cuda*) in FFmpeg for scaling the decoded video output into multiple desired resolutions. Doing this ensures that the memory transfers (system memory to video memory and vice versa) are eliminated, and that transcoding is performed with the highest possible performance on the GPU hardware.

Input: *input.mp4*

Outputs: *1080p*, *720p* (audio same as input)

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
  -vf scale_cuda=1920:1080:interp_algo=bilinear -c:a copy -c:v h264_nvenc -
  ↪b:v 5M -fps_mode passthrough output1.mp4 \
  -vf scale_cuda=1280:720:interp_algo=bilinear -c:a copy -c:v h264_nvenc -
  ↪b:v 8M -fps_mode passthrough output2.mp4
```

4.5. 1:N Encode from YUV or RAW Data

Encoding from YUV or RAW files can result in disk I/O being a bottleneck, and it is advised to do such encodes from an SSD to get maximum performance. The following command reads file *input.yuv* and encodes it to four different H.264 videos at various output bit rates. Note that this command results in a single YUV load only for all encode operations, resulting in more efficient disk I/O to improve the overall encode performance.

Input: *input.yuv* (*yuv420p*, *1080p*)

Outputs: *1080p (8M)*, *1080p (10M)*, *1080p (12M)*, *1080p (14M)*

```
ffmpeg -y -f rawvideo -pix_fmt yuv420p -s 1920x1080 -i input.yuv \
  -filter_complex "[0:v]hwupload_cuda,split=4[o1][o2][o3][o4]" \
  -map "[o1]" -c:v h264_nvenc -b:v 8M -fps_mode passthrough output1.mp4 \
  -map "[o2]" -c:v h264_nvenc -b:v 10M -fps_mode passthrough output2.mp4 \
  -map "[o3]" -c:v h264_nvenc -b:v 12M -fps_mode passthrough output3.mp4 \
  -map "[o4]" -c:v h264_nvenc -b:v 14M -fps_mode passthrough output4.mp4
```

Note

Other pixel formats are also supported by NVENC. For example, the pixel format (*pix_fmt*) should be changed to *yuv444p*, *p010*, or *p210* for encoding YUV 4:4:4 8-bit, 4:2:0 10-bit, and 4:2:2 10-bit files respectively.

4.6. Multiple 1:N Transcode with Scaling

This method should be used to realize the full potential of GPU hardware-accelerated transcoding. One of the typical workloads for transcoding consists of videos being transcoded and archived at different resolutions and bitrates so that they can be served to different clients later. The following command reads file *input1.mp4* as the input, decodes it in GPU hardware, scales the input in hardware,

and re-encodes as H.264 videos to *output11.mp4* at 480p and *output12.mp4* at 240p using the GPU hardware encoder. Simultaneously it reads file *input2.mp4* and transcodes it to *output21.mp4* at 720p and *output22.mp4* at 480p as H.264 videos. These are achieved using a single command line.

Input: *input1.mp4*, *input2.mp4*

Output: 480p, 240p (from *input1.mp4*), 720p, 480p (from *input2.mp4*) (video only)

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input1.mp4 \
    -hwaccel cuda -hwaccel_output_format cuda -i input2.mp4 \
    -map 0:0 -vf scale_cuda=640:480:interp_algo=bilinear -c:v h264_nvenc -b:v
    ↪1M output11.mp4 \
    -map 0:0 -vf scale_cuda=320:240:interp_algo=bilinear -c:v h264_nvenc -b:v
    ↪500k output12.mp4 \
    -map 1:0 -vf scale_cuda=1280:720:interp_algo=bilinear -c:v h264_nvenc -
    ↪b:v 3M output21.mp4 \
    -map 1:0 -vf scale_cuda=640:480:interp_algo=bilinear -c:v h264_nvenc -b:v
    ↪2M output22.mp4
```

4.7. Multiple 1:N Transcode with Scaling (SW Decode->HW Scaling->HW Encode)

In some situations, it is necessary to perform video decoding in software. For example, when the input codec is not supported by the hardware decoder, or when the hardware encoder has significantly more capacity than the decoder, software decoding can be used while still leveraging GPU acceleration for scaling and encoding.

The following command reads file *input1.mp4*, decodes it in software, uploads frames to GPU memory, scales them using hardware acceleration, and encodes to *output11.mp4* at 480p and *output12.mp4* at 240p as H.264 videos. Simultaneously it reads file *input2.mp4* and transcodes it to *output21.mp4* at 720p and *output22.mp4* at 480p as H.264 videos using the same software decode -> hardware scale -> hardware encode workflow.

Input: *input1.mp4*, *input2.mp4*

Output: 480p, 240p (from *input1.mp4*), 720p, 480p (from *input2.mp4*) (video only)

```
ffmpeg -y -init_hw_device cuda=foo:0 -filter_hw_device foo \
    -i input1.mp4 -i input2.mp4 \
    -map 0:0 -vf hwupload,scale_cuda=640:480:interp_algo=bilinear -c:v h264_
    ↪nvenc -b:v 1M \
        output11.mp4 \
    -map 0:0 -vf hwupload,scale_cuda=320:240:interp_algo=bilinear -c:v h264_
    ↪nvenc -b:v 500k \
        output12.mp4 \
    -map 1:0 -vf hwupload,scale_cuda=1280:720:interp_algo=bilinear -c:v h264_
    ↪nvenc -b:v 2M \
        output21.mp4 \
    -map 1:0 -vf hwupload,scale_cuda=640:480:interp_algo=bilinear -c:v h264_
    ↪nvenc -b:v 1M \
        output22.mp4
```


Note

Since this command uses software decoding (no `-hwaccel cuda` before inputs), the CUDA device context must be explicitly created:

- ▶ `-init_hw_device cuda=foo:0` creates a CUDA device context named “foo” on GPU device 0 (use 1, 2, etc. for other GPUs)
- ▶ `-filter_hw_device foo` tells all filters to use the CUDA device context named “foo”

These options enable the `scale_cuda` filter and `h264_nvenc` encoder to use GPU device 0 for hardware acceleration.

Chapter 5. Video Decoding

CUVID is both a **parser** and a **decoder**: it parses the compressed bitstream and decodes it on the NVDEC hardware. Because CUVID handles parsing itself, FFmpeg can use NVDEC in two ways: (1) through the CUVID decoders (h264_cuvid, hevc_cuvid, etc.), which use CUVID for both parsing and decoding, or (2) through NVDEC hardware acceleration (-hwaccel nvdec / -hwaccel cuda), which uses FFmpeg's standard parsers and only offloads decoding to NVDEC. Both approaches use the same NVDEC hardware; they differ in who does the parsing (CUVID vs. FFmpeg).

Decoding Approaches

FFmpeg provides two methods for hardware-accelerated video decoding:

1. **CUVID Decoders** (h264_cuvid, hevc_cuvid, av1_cuvid, etc.)
 - ▶ Standalone decoders that include both an NVIDIA bitstream parser and the NVDEC decoder
 - ▶ Provide decoder-specific options like -resize and -crop for GPU-accelerated scaling/cropping during decode
 - ▶ Require explicit codec specification (e.g., h264_cuvid for H.264)
 - ▶ **Supported codecs**: H.264, HEVC, AV1, VP8, VP9, MPEG-1, MPEG-2, MPEG-4, VC1, MJPEG
2. **NVDEC Hardware Acceleration** (-hwaccel nvdec or -hwaccel cuda) - **Recommended**
 - ▶ Hardware acceleration layer that works with FFmpeg's standard parsers
 - ▶ Automatic codec detection
 - ▶ More flexible for complex filter graphs
 - ▶ Integrates better with FFmpeg's hwaccel framework
 - ▶ Preferred approach for most use cases
 - ▶ **Supported codecs**: Same as CUVID (both use the same NVDEC hardware)

Basic Decoding - CUVID Decoder

To decode an input H.264 bitstream from *input.mp4* to raw YUV using CUVID decoder:

```
ffmpeg -y -c:v h264_cuvid -i input.mp4 output.yuv
```

Basic Decoding - NVDEC Hardware Acceleration

To decode with automatic codec detection using NVDEC hwaccel:

```
ffmpeg -y -hwaccel cuda -i input.mp4 output.yuv
```

Both methods generate output files in NV12 format (*output.yuv*) for yuv420p input. Frames are automatically transferred to system memory for file output.

Decoding with GPU-Accelerated Resize (CUVID only)

CUVID decoders provide built-in GPU-accelerated resize and crop during decode:

```
ffmpeg -y -c:v h264_cuvid -resize 1280x720 -i input.mp4 output.yuv
```

```
ffmpeg -y -c:v h264_cuvid -crop 16x16x32x32 -i input.mp4 output.yuv
```

Concurrent Decoding

To decode multiple input bitstreams concurrently within a single FFmpeg process (useful for maximizing decoder throughput):

```
ffmpeg -y \  
  -hwaccel cuda -i input1.mp4 \  
  -hwaccel cuda -i input2.mp4 \  
  -hwaccel cuda -i input3.mp4 \  
  -map 0:v -f rawvideo output1.yuv \  
  -map 1:v -f rawvideo output2.yuv \  
  -map 2:v -f rawvideo output3.yuv
```

This uses a separate thread per decode operation with a single CUDA context shared among all threads.

Chapter 6. Video Encoding

NVENC provides extensive control over encoding parameters to optimize for different use cases. Understanding these parameters and their trade-offs is essential for achieving the desired balance between quality, latency, performance, and memory usage.

This section explains the key NVENC parameters and provides recommended command-line configurations for common use cases based on [NVIDIA Video Codec SDK documentation](#).

6.1. Understanding Key NVENC Parameters

The following parameters significantly impact encoding quality, performance, and latency:

6.1.1. Presets

NVENC presets control the speed/quality trade-off of the encoding process. Higher-numbered presets produce better quality output but require more processing time.

- ▶ `-preset p1`: Fastest encoding, lowest quality (legacy HP/High Performance preset)
- ▶ `-preset p2`: Faster encoding, lower quality
- ▶ `-preset p3`: Fast encoding, low quality
- ▶ `-preset p4`: Medium encoding speed and quality (default, legacy Default preset)
- ▶ `-preset p5`: Slow encoding, good quality (legacy BD preset)
- ▶ `-preset p6`: Slower encoding, better quality (legacy HQ preset)
- ▶ `-preset p7`: Slowest encoding, best quality

FFmpeg also provides convenient aliases: `-preset fast` (maps to p1), `-preset medium` (maps to p4, single pass), and `-preset slow` (maps to p7, two-pass).

6.1.2. Tuning Info

The `-tune` option optimizes encoder behavior for specific use cases:

- ▶ `-tune uhq`: Ultra high quality mode (enables lookahead and temporal filter for maximum quality, higher memory usage). Only supported for HEVC and AV1 on Turing+ architectures
- ▶ `-tune hq`: High quality mode for latency-tolerant scenarios (VOD, archiving)
- ▶ `-tune ll`: Low latency mode for interactive applications (reduces buffering)

- ▶ `-tune ull`: Ultra-low latency mode for real-time streaming (minimal buffering)
- ▶ `-tune lossless`: Lossless encoding mode

6.1.3. Rate Control Modes

Rate control determines how the encoder allocates bits across frames:

- ▶ **CONSTQP (Constant QP)**: `-rc constqp` - Constant quantization parameter mode, used with `-qp` to specify QP values for I/P/B frames
- ▶ **VBR (Variable Bitrate)**: `-rc vbr` - Variable bitrate mode; control via `-b:v` (target bitrate), `-maxrate` (maximum bitrate), and `-bufsize` (VBV buffer size). Bitrate varies with content complexity.
- ▶ **VBR-CQ (Variable Bitrate, Constant Quality)**: `-rc vbr` with `-cq N` - Targets a constant quality level instead of a target bitrate; control via `-cq N` (0-51 for H.264/HEVC, 0-63 for AV1; lower is better quality; 0 means automatic) and `-maxrate` (peak bitrate cap).
- ▶ **CBR (Constant Bitrate)**: `-rc cbr` - Maintains constant bitrate, required for streaming applications with strict bandwidth constraints

6.1.4. B-frames and Reference Frames

- ▶ `-bf N`: Number of bidirectional B-frames between I/P frames (0-5). Bidirectional B-frames reference both past and future frames, improving compression efficiency but increasing latency due to frame reordering
- ▶ `-b_ref_mode [disabled|each|middle]`: Controls whether B-frames can be used as references for other frames, improving quality at the cost of memory
 - ▶ `disabled`: B-frames are not used for reference
 - ▶ `each`: Each B-frame will be used for reference
 - ▶ `middle`: For H.264/HEVC, only the middle B-frame is used for reference: the $(N/2)$ th B-frame where N = number of B-frames (for odd N , the $(N-1)/2$ -th B-frame). For AV1, every other B-frame is set as an Altref2 reference except the last B-frame in the Altref interval.
- ▶ `-unidir_b 1`: Enable unidirectional B-frames where both references are from the past. **When enabled, NVENC replaces ALL P-frames with unidirectional B-frames** that have no reordering delay. For low-latency use cases, combine with `-bf 0` to disable bidirectional B-frames entirely (pattern: IBBBBB with only unidirectional B-frames). Provides 5-10% better compression than P-only encoding with no additional latency. Particularly useful for live streaming and real-time applications. Requires FFmpeg 7.1+
- ▶ `-nonref_p 1`: Enable automatic insertion of non-reference P-frames. These P-frames are not used as references for encoding future frames. **Benefits**: Provides error resilience for unreliable networks - if a non-reference P-frame is lost during transmission, it won't propagate errors to subsequent frames.
- ▶ `-weighted_pred 1`: Enable weighted prediction. Improves compression efficiency (1-3% bitrate savings) for content with fades, dissolves, and lighting changes by analyzing temporal brightness variations between frames. **Not compatible with B-frames** - requires `-bf 0`. Only beneficial for P-frame-only encoding scenarios

6.1.5. GOP (Group of Pictures) Structure

- ▶ `-g N`: GOP size - distance between I-frames. Typical values: 250 (10 seconds at 25fps), 999999 (very large GOP for low-latency to avoid periodic I-frame spikes). Note: `-g -1` uses the preset's default GOP size (typically 250), not infinite
- ▶ `-intra-refresh 1`: Enable intra refresh mode (Gradual Decoder Refresh/GDR) as an alternative to periodic I-frames. Instead of encoding full I-frames at regular intervals (which causes bitrate spikes), intra refresh gradually refreshes the video by encoding different regions as intra macroblocks over multiple frames. This provides consistent bitrate, lower latency, and better error resilience for live streaming. Requires `-g N` to specify the refresh period.
- ▶ `-single-slice-intra-refresh 1`: Maintain single slice during intra refresh (reduces overhead, requires intra refresh to be enabled)

The combination of `-g` (GOP size) and `-bf` (B-frames) determines the frame pattern:

- ▶ **IPPPP...** (P-frames only, large GOP): `-g 999999 -bf 0` - For scenarios with very infrequent I-frames
- ▶ **IPPPP...** (P-frames with intra refresh): `-intra-refresh 1 -g 250 -bf 0` - Infinite GOP with gradual refresh instead of periodic I-frames, ideal for low-latency streaming with consistent bitrate
- ▶ **IBBBBB...** (Unidirectional B-frames with intra refresh): `-intra-refresh 1 -g 250 -bf 0 -unidir_b 1` - **Recommended for low-latency**: P-frames are replaced with unidirectional B-frames that reference only past frames, providing better compression than P-only with no additional latency
- ▶ **IBBBBPBBBBP...** (B-frames between P-frames): `-g 250 -bf 4` - Maximum compression efficiency at the cost of latency
- ▶ **IIIII...** (All I-frames): `-g 0` - Highest quality and editing flexibility, but largest file size

6.1.6. Bitrate Control

- ▶ `-b:v N`: Target video bitrate (e.g., 5M for 5 Mbps). This is the average bitrate the encoder aims to achieve
- ▶ `-maxrate N`: Maximum bitrate allowed (peak bitrate cap):
 - ▶ `-maxrate = -b:v`: Enforces constant bitrate (CBR) behavior. Use for live streaming, video conferencing, and bandwidth-constrained scenarios where predictable bitrate is critical
 - ▶ `-maxrate > -b:v`: Allows variable bitrate (VBR). Bitrate can spike up to maxrate for complex scenes while averaging at `-b:v`. Better quality for VOD/transcoding
- ▶ `-bufsize N`: Video Buffering Verifier (VBV) buffer size in bits (not bits/sec). Controls how much bitrate can vary over time. Buffer duration = `bufsize / maxrate`:
 - ▶ Very large buffers (e.g., `-bufsize 20M -maxrate 5M` = 4 second buffer): **Recommended by NVIDIA for recording/archiving and high-quality transcoding**. Allows maximum quality optimization for complex scenes
 - ▶ Large buffers (e.g., `-bufsize 10M -maxrate 5M` = 2 second buffer): Good for quality-focused VOD/OTT streaming
 - ▶ Medium buffers (e.g., `-bufsize 5M -maxrate 5M` = 1 second buffer): Standard live streaming scenarios

- ▶ Small buffers (e.g., `-bufsize 167K -maxrate 5M` = 1 frame at 30fps): **Recommended by NVIDIA for ultra-low latency** (game streaming, video conferencing). Strict frame-by-frame bitrate control
- ▶ `-qmin N` and `-qmax N`: Minimum and maximum quantization parameters that constrain the encoder's QP range. Valid range: 0-51 for H.264/HEVC, 0-255 for AV1 (useful for quality control in VBR/CBR modes)

6.1.7. Lookahead

- ▶ `-rc-lookahead N`: Number of frames to buffer for rate control analysis. **Maximum depth is (31 - number of B-frames). If a higher value is specified, it will be clamped by the driver to this maximum**
 - ▶ Improves rate-control accuracy by analyzing future frames before encoding, particularly for adaptive I-frame insertion at scene cuts and adaptive B-frame decision
 - ▶ Lookahead allocates additional buffers in video memory for frames buffered in the lookahead queue, increasing memory usage proportional to lookahead depth
 - ▶ Automatically enabled by some presets (P6, P7) for better quality
 - ▶ For memory-constrained scenarios, disable lookahead (`-rc-lookahead 0`) to reduce video memory footprint
 - ▶ Increases encoding latency as input frames must remain available to the encoder until encode completion
- ▶ `-lookahead_level [auto|0|1|2|3]`: Specifies the lookahead level. Higher level may improve quality at the expense of performance (default: auto)

6.1.8. Adaptive Quantization (AQ)

Adaptive Quantization adjusts encoding quantization parameters based on frame characteristics to improve perceptual quality.

Spatial AQ

- ▶ `-spatial-aq 1`: Enables spatial adaptive quantization
- ▶ `-aq-strength N`: Controls QP variation intensity (1 (least aggressive) to 15 (most aggressive), default: 8)
- ▶ Allocates more bits to flat/smooth regions (more perceptually important) at the cost of detailed regions
- ▶ **Improves perceptual quality but may reduce PSNR metrics (disable this option for PSNR evaluation)**
- ▶ Recommended for second-generation Maxwell GPUs and newer

Temporal AQ

- ▶ `-temporal-aq 1`: Enables temporal adaptive quantization
- ▶ Allocates more bits to static high-detail regions to improve reference frame quality
- ▶ Most beneficial for content with static detailed backgrounds and low-motion foregrounds
- ▶ **May cause frame size fluctuation within GOP (not recommended for strict per-frame CBR requirements)**

- Recommended for second-generation Maxwell GPUs and newer

Warning

Do not enable spatial and temporal adaptive quantization together. Use either spatial AQ or temporal AQ, not both. Using both can lead to conflicting bit allocation and is not recommended.

6.1.9. Temporal Filtering

Temporal filtering is a noise reduction technique that analyzes multiple frames to reduce temporal noise while preserving detail.

- `-tf_level [0|4]`: Specifies the strength of temporal filtering. Supported values:
 - 0: Temporal filtering disabled
 - 4: Temporal filtering enabled with strength level 4
- **Requirements:**
 - Requires at least 4 B-frames (`-bf 4`) - temporal filter needs `frameIntervalP >= 5`
 - Not compatible with zero reorder delay (ultra-low latency mode)
 - Not compatible with stereo Multiview Video Coding encoding
 - Not compatible with alpha layer encoding (HEVC/AV1)
- **Recommended for:** Natural content (real-world video, not synthetic/animated content)
- **Benefits:** Reduces temporal noise and improves visual quality, particularly for content captured with noisy sensors or in low-light conditions
- **Availability:** Supported for H.264, HEVC, and AV1 on compatible hardware
- **Note:** Automatically enabled by UHQ tuning (`-tune uhq`)

6.1.10. Multi-Pass Encoding

For NVENC-based encoders (`*_nvenc`), the `-multipass` option controls per-frame multi-pass rate control inside NVENC, not classic file-level 2-pass encoding. NVENC still runs as a single encode job; it just does one or two internal passes for each frame to improve bit allocation.

- `-multipass [disabled|qres|fullres]`:
 - `disabled` (`NV_ENC_MULTI_PASS_DISABLED`): Single-pass encoding. Each frame is analyzed and encoded once; lowest latency and memory use, but rate control has less information about frame complexity.
 - `qres` (Quarter Resolution, `NV_ENC_TWO_PASS_QUARTER_RESOLUTION`): Two-pass encoding per frame — first pass at quarter resolution, second at full resolution. The quarter-res pass is cheaper and can help detect large motion vectors while keeping GPU cost lower, but the statistics are somewhat coarser.
 - `fullres` (Full Resolution, `NV_ENC_TWO_PASS_FULL_RESOLUTION`): Two-pass encoding per frame with both passes at full resolution. Most accurate frame-complexity and motion statistics, allowing NVENC to place bits more precisely, at the cost of additional encoding time and higher video-memory usage.

In general, multi-pass modes improve rate control accuracy (especially for CBR / tight VBV) and can bring the actual bitrate closer to the target, but they do so at the expense of encoding time and memory. Use them when quality and bitrate accuracy matter more than minimum latency or peak throughput.

6.1.11. Split Frame Encoding

- ▶ `-split_encode_mode [disabled|auto|forced|2|3|4]`: Controls split frame encoding for parallel processing on GPUs with multiple NVENC engines. Available for HEVC and AV1 (not H.264)
 - ▶ **disabled**: Split frame encoding disabled for all configurations
 - ▶ **auto**: Enabled or disabled automatically depending on preset and tuning (default)
 - ▶ **forced**: Enabled with the driver selecting the optimal number of horizontal strips automatically
 - ▶ **2**: Force 2-way horizontal split when multiple NVENC engines are available
 - ▶ **3**: Force 3-way horizontal split when multiple NVENC engines are available (requires 3 NVENC engines)
 - ▶ **4**: Force 4-way horizontal split when multiple NVENC engines are available (requires 4 NVENC engines)
- ▶ **How it works**: Divides each frame into horizontal strips that are encoded in parallel across multiple NVENC engines, improving throughput with minimal quality loss
- ▶ **Best for**: High-resolution encoding (4K, 8K) where encoding speed is critical
- ▶ **Requirements**: GPU must have multiple NVENC engines. The [Video Encode and Decode Support Matrix](#) lists the number of NVENC (and NVDEC) engines per GPU
- ▶ **Recommendation**: Use forced to let the driver decide optimal split configuration, or use 2 if you know your GPU has 2 NVENC engines

For more on split-frame encoding and high-resolution workflows (e.g. 8K60), see the [NVIDIA Developer Blog: Video encoding at 8K60 with split-frame encoding and NVIDIA Ada Lovelace architecture](#).

6.2. Use Case Recommendations

Based on NVIDIA Video Codec SDK recommendations, the following configurations are optimized for specific use cases. Treat these as **general guidelines or a starting point**—you can modify any of the settings to suit your content, quality targets, and constraints; they are not meant as ground truth for every scenario.

Note

About the Examples: All examples demonstrate **file-to-file transcoding** (input.mp4 -> output.mp4) to illustrate the encoder settings. For real-time capture (game recording, screen capture) or live streaming (game casting, broadcasting), you will need to use appropriate input sources and output destinations, but the recommended NVENC encoding parameters remain the same.

6.2.1. Recording and Archiving

For recording and archiving scenarios where maximum quality is critical and encoding can be done offline or with relaxed real-time constraints.

Recommended Settings (Bitrate-Based VBR):

NVIDIA's official recommendation for recording/archiving uses bitrate-based VBR with:

- ▶ Slower preset (p6 or p7) with high-quality or ultra-high-quality tuning
- ▶ Variable bitrate (VBR) rate control with high target bitrate
- ▶ **Very large VBV buffer size (4 seconds)** - Allows maximum quality optimization for complex scenes
- ▶ Lookahead enabled
- ▶ B-frames enabled with B-frames as reference
- ▶ **Adaptive quantization for non-objective quality evaluation** (spatial **or** temporal AQ only—do not enable both)
- ▶ Multi-pass encoding (quarter or full resolution)
- ▶ Optional: Temporal filtering for natural content

Example - Ultra-High-Quality Recording (4K60):

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
  -c:a copy -c:v hevc_nvenc \
  -preset p7 -tune uhq -multipass fullres \
  -rc vbr -b:v 40M -maxrate 60M -bufsize 240M \
  -bf 4 -b_ref_mode each \
  -spatial-aq 1 -aq-strength 10 \
  -fps_mode passthrough output.mp4
```

Note

4-second VBV buffer (`-bufsize 240M = 60M × 4 seconds`) gives the encoder maximum flexibility to optimize quality for complex scenes while maintaining the target average bitrate (`-b:v 40M`). This matches NVIDIA's official recommendation for recording/archiving workflows.

With `-tune uhq`, lookahead and temporal filtering are automatically enabled; you do not need to set `-rc-lookahead` or `-tf_level` explicitly.

Warning

`-spatial-aq 1 -aq-strength 10` should only be used when quality tests (e.g. PSNR or other objective metrics) are not going to be performed. Spatial adaptive quantization improves perceptual quality only and may reduce PSNR; disable it for objective quality evaluation.

6.2.2. Game and Studio Broadcasting

For game and studio broadcasting where good quality is needed with reasonable encoding performance.

Recommended Settings:

- ▶ Medium to slow presets (p4-p6) with high-quality or ultra-high-quality tuning
- ▶ **Constant bitrate (CBR)** rate control
- ▶ **Medium VBV buffer size (1 second)** - bufsize = maxrate = bitrate
- ▶ Lookahead enabled
- ▶ B-frames enabled with B-frames as reference
- ▶ **Spatial adaptive quantization for non-objective quality evaluation** (temporal AQ should be disabled for CBR)
- ▶ Multi-pass encoding (quarter resolution for performance balance)
- ▶ Finite GOP length (approximately 2 seconds)

Example - 1080p60:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
-preset p5 -tune hq -multipass qres \
-rc cbr -b:v 10M -maxrate 10M -bufsize 10M \
-bf 3 -b_ref_mode middle -g 120 \
-rc-lookahead 16 \
-fps_mode passthrough output.mp4
```

Example - 4K60:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
-preset p5 -tune hq -multipass qres \
-rc cbr -b:v 40M -maxrate 40M -bufsize 40M \
-bf 3 -b_ref_mode middle -g 120 \
-rc-lookahead 20 \
-fps_mode passthrough output.mp4
```

Note

CBR with 1-second VBV buffer: NVIDIA's official recommendation uses CBR (-bufsize = -maxrate = -b:v) for predictable bandwidth and performance in real-time game and studio broadcasting. The 1-second buffer allows some bitrate flexibility while maintaining overall rate consistency.

6.2.3. Low-Latency Streaming and Real-Time Applications

For interactive applications such as game streaming, live broadcasting, video conferencing, and remote desktop where latency must be minimized (16-100ms):

Recommended Settings:

- ▶ Fast preset (p2) or medium preset (p4) with ultra-low-latency or low-latency tuning
- ▶ **Constant bitrate (CBR)** rate control
- ▶ **Multi-pass encoding** (quarter or full resolution - evaluate based on quality/performance tradeoff)
- ▶ **Very low VBV buffer size** (single frame: bufsize = bitrate / framerate)

- **Unidirectional B-frames** (`-bf 0 -unidir_b 1`) for better compression with no latency penalty
- Very large or infinite GOP length (`-g 999999` or `-intra-refresh 1 -g 250`) to avoid periodic I-frame spikes
- **Spatial adaptive quantization for non-objective quality evaluation** (temporal AQ should be disabled for CBR)
- Optional: **Intra refresh** (`-intra-refresh 1`), **non-reference P frames** (`-nonref_p 1`) for error resilience

Example - Ultra-Low Latency with Non-Reference P Frames (1080p30):

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
-preset p2 -tune ull \
-rc cbr -b:v 5M -bufsize 167K -maxrate 5M \
-bf 0 -unidir_b 1 -g 999999 -nonref_p 1 \
-fps_mode passthrough output.mp4
```

Example - Low Latency with Unidirectional B-frames (Recommended, 1080p30):

For improved compression efficiency while maintaining true low latency (requires FFmpeg 7.1+):

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
-preset p4 -tune ll -multipass qres \
-rc cbr -b:v 5M -bufsize 167K -maxrate 5M \
-bf 0 -unidir_b 1 -g 999999 \
-fps_mode passthrough output.mp4
```

This configuration uses unidirectional B-frames (IBBBBBB pattern) for 5-10% better compression than P-only encoding with no frame reordering delay, multi-pass encoding for better rate control, and infinite GOP to avoid periodic I-frame bitrate spikes.

Example - Low Latency High Quality with Intra Refresh (1080p30):

For better quality with acceptable latency and improved error resilience:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
-preset p6 -tune ll -multipass qres \
-rc cbr -b:v 5M -bufsize 167K -maxrate 5M \
-bf 0 -unidir_b 1 -intra-refresh 1 -g 250 \
-fps_mode passthrough output.mp4
```

This configuration demonstrates **intra refresh** (`-intra-refresh 1 -g 250`), which gradually refreshes the video with intra-coded macroblocks instead of periodic full I-frames. This avoids bitrate spikes while providing error resilience for unreliable networks.

Note

CBR with 1-frame VBV buffer: The VBV buffer size for single-frame buffering can be calculated as: $\text{bufsize} = \text{bitrate} / \text{framerate}$. For example, at 5 Mbps and 30 fps: $\text{bufsize} = 5000000 / 30 \approx 167\text{K}$.

6.2.4. Lossless Encoding

For applications requiring bit-exact reproduction of the input (archival, intermediate formats, mastering):

```
ffmpeg -y -f rawvideo -s 1920x1080 -pix_fmt yuv420p -i input.yuv \
-c:v hevc_nvenc -preset p4 -tune lossless \
-fps_mode passthrough output.mp4
```

Note

Why preset matters for lossless: Even though lossless mode always produces bit-exact output regardless of preset, the preset still affects:

- ▶ **Encoding speed:** Faster presets (p1-p3) encode quicker; slower presets (p4-p7) take longer
- ▶ **Compression efficiency:** Slower presets produce smaller file sizes through more thorough analysis, even though quality is identical
- ▶ **B-frame usage:** Presets p3 and higher automatically enable B-frames in lossless mode for better compression

Recommendation: Use p4-p5 for balanced speed and compression. Use p7 for maximum compression when encoding time is not a concern. Lossless encoding produces significantly larger files than lossy encoding. HEVC lossless typically achieves better compression ratios than H.264 lossless.

6.2.5. Memory-Constrained Encoding

For scenarios with limited GPU memory (embedded systems, multi-session encoding), the following guidelines reduce memory footprint:

Recommended Settings:

- ▶ Disable or reduce B-frames (-bf 0 or -bf 1)
- ▶ Reduce maximum reference frames (-dpb_size 1 or -dpb_size 2) - Controls DPB (Decoded Picture Buffer) size
- ▶ Use single-pass rate control (-multipass disabled)
- ▶ Disable adaptive quantization (-spatial-aq 0 -temporal-aq 0)
- ▶ Disable lookahead (-rc-lookahead 0)
- ▶ Disable weighted prediction (-weighted_pred 0)
- ▶ Disable temporal filter (-tf_level 0)
- ▶ Avoid UHQ tuning info (it enables lookahead and temporal filter)

Example:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
-preset p4 -tune hq -multipass disabled \
-bf 0 -dpb_size 1 -rc-lookahead 0 \
-spatial-aq 0 -temporal-aq 0 -weighted_pred 0 -tf_level 0 \
```

(continues on next page)

(continued from previous page)

```
-b:v 5M \
-fps_mode passthrough output.mp4
```

Warning

Memory reduction settings will negatively impact encoding quality and compression efficiency. Use only when memory constraints are critical.

6.2.6. High-Quality Live Streaming

For live streaming scenarios where quality is more important than absolute minimal latency (e.g., content creation, professional broadcasting), use higher presets with optimized settings for smooth playback:

Recommended Settings:

- ▶ High-quality preset (p6 or p7) with high-quality or ultra-high-quality tuning
- ▶ CBR rate control for consistent bandwidth
- ▶ B-frames for better compression
- ▶ Moderate VBV buffer (1-2 seconds)
- ▶ Adaptive quantization for non-objective quality evaluation
- ▶ GOP aligned with segment duration (typically 2 seconds)

Example - 4K60 High-Quality Stream:

For ultra-high-definition live streaming:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
-preset p7 -tune hq -multipass qres \
-rc cbr -b:v 20M -bufsize 40M -maxrate 20M \
-bf 4 -b_ref_mode middle -g 120 \
-rc-lookahead 16 -lookahead_level 2 \
-split_encode_mode forced \
-fps_mode passthrough output.mp4
```

Note

HEVC is recommended for 4K60 streaming for better compression efficiency over H.264. AV1 provides even better compression but check platform compatibility.

6.2.7. Multi-Stream Adaptive Bitrate (ABR) Ladder

Modern streaming platforms require multiple resolution/bitrate renditions for adaptive streaming. This example demonstrates creating a 6-stream ABR ladder from a single 4K60 input.

Example - 6-Stream ABR Ladder:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
  -filter_complex "\
    [0:v]split=6[base][s1][s2][s3][s4][s5]; \
    [base]scale_cuda=3840:2160:interp_algo=bilinear[v0]; \
    [s1]scale_cuda=2560:1440:interp_algo=bilinear[v1]; \
    [s2]scale_cuda=1920:1080:interp_algo=bilinear[v2]; \
    [s3]scale_cuda=1280:720:interp_algo=bilinear[v3]; \
    [s4]scale_cuda=854:480:interp_algo=bilinear[v4]; \
    [s5]scale_cuda=640:360:interp_algo=bilinear[v5]" \
  -map "[v0]" -c:v hevc_nvenc -preset p6 -tune hq -b:v 25M -maxrate 25M -
  ↪ bufsize 50M -g 120 -bf 2 output_4k60.mp4 \
  -map "[v1]" -c:v hevc_nvenc -preset p6 -tune hq -b:v 12M -maxrate 12M -
  ↪ bufsize 24M -g 120 -bf 2 output_1440p60.mp4 \
  -map "[v2]" -c:v hevc_nvenc -preset p6 -tune hq -b:v 8M -maxrate 8M -
  ↪ bufsize 16M -g 120 -bf 2 output_1080p60.mp4 \
  -map "[v3]" -c:v hevc_nvenc -preset p6 -tune hq -b:v 5M -maxrate 5M -
  ↪ bufsize 10M -g 120 -bf 2 output_720p60.mp4 \
  -map "[v4]" -c:v hevc_nvenc -preset p6 -tune hq -b:v 2M -maxrate 2M -
  ↪ bufsize 4M -g 60 -bf 2 -r 30 output_480p30.mp4 \
  -map "[v5]" -c:v hevc_nvenc -preset p6 -tune hq -b:v 1M -maxrate 1M -
  ↪ bufsize 2M -g 60 -bf 2 -r 30 output_360p30.mp4
```

6.2.8. Video Editing Workflows

Video editing applications require different encoding strategies depending on the export scenario. This section covers the most common workflows.

Ultra-High Quality Exports

For final deliverables where quality is paramount (film, broadcast, archival):

Example - 4K60 UHQ Export:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
  -c:a copy -c:v hevc_nvenc \
  -preset p7 -tune uhq \
  -rc vbr -cq 19 -maxrate 80M \
  -bf 5 -b_ref_mode middle -g 120 \
  -rc-lookahead 26 \
  -multipass fullres \
  -fps_mode passthrough output.mp4
```

Note

UHQ tuning (`-tune uhq`) is only supported for HEVC and AV1 on Turing and newer architectures. It enables lookahead and temporal filtering automatically for maximum quality. This mode requires significantly more GPU memory.

Fast Export

For quick previews, proxies, or when speed is critical:

Example - 4K60 Fast Export:


```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
-preset p1 -tune hq \
-rc vbr -cq 23 -maxrate 30M \
-bf 0 -g 120 \
-fps_mode passthrough output.mp4
```

Split Frame Encoding (Parallel Processing)

For maximizing throughput on GPUs with multiple NVENC engines or using split-frame encoding:

Example - 4K60 with 3-Way Split Frame:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
-preset p4 -tune hq \
-rc vbr -cq 20 -maxrate 40M \
-bf 2 -g 120 \
-split_encode_mode 3 \
-multipass disabled \
-fps_mode passthrough output.mp4
```

6.2.9. High-Resolution Encoding (8K)

Ultra-high-resolution 8K content requires careful parameter tuning for optimal quality and performance.

8K60 General Guidelines:

- ▶ HEVC or AV1 strongly recommended (H.264 is inefficient at 8K)
- ▶ Significantly higher bitrates (80-150 Mbps for HEVC)
- ▶ May require multiple NVENC engines with split encoding for real-time performance
- ▶ Ensure sufficient GPU memory (8GB+ recommended)
- ▶ Consider reducing preset if real-time encoding is required

Example - 8K60 HEVC High Quality (VOD/Offline):

For offline encoding or VOD where quality is paramount:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
-preset p5 -tune hq \
-rc vbr -cq 19 -maxrate 180M \
-bf 3 -b_ref_mode middle -g 120 \
-rc-lookahead 16 \
-fps_mode passthrough output.mp4
```

Example - 8K60 HEVC Live Streaming:

For real-time 8K streaming where encoding performance is critical:

```
ffmpeg -y -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \
-c:a copy -c:v hevc_nvenc \
```

(continues on next page)

(continued from previous page)

```
-preset p3 -tune hq -multipass qres \  
-rc cbr -b:v 100M -maxrate 100M -bufsize 200M \  
-bf 5 -b_ref_mode middle -g 120 \  
-rc-lookahead 12 -lookahead_level 1 \  
-split_encode_mode forced \  
-fps_mode passthrough output.mp4
```

Note

8K Streaming Configuration:

- ▶ **CBR with 2-second buffer** (-bufsize 200M = 100M × 2) for consistent bandwidth
- ▶ **Preset p3** for real-time performance balance (p4-p7 may be too slow for real-time 8K60)
- ▶ **Multi-pass quarter resolution** for better rate control without excessive overhead
- ▶ **Split encoding** to utilize multiple NVENC engines for improved throughput
- ▶ **Reduced lookahead** (12 frames) to minimize latency and memory usage
- ▶ **Bitrate:** 100 Mbps is conservative for 8K60; increase to 120-150 Mbps for complex content

Chapter 7. Performance Evaluation and Optimization

Various factors affect the performance of hardware accelerated transcoding on the GPU. Getting the highest performance for your workload requires some tuning. This section provides some tips for measuring and optimizing end-to-end transcode performance.

NVIDIA Video Codec SDK documentation publishes performance of GPU hardware accelerated encoder and decoder as stand-alone numbers, measured using high-performance encode or decode application included in the SDK. Although FFmpeg software is highly optimized, its performance is slightly lower than the performance reported in the SDK documentation, mainly due to software overheads and additional setup/initialization time within FFmpeg code. Therefore, to get high transcoding throughput using FFmpeg, it is essential to saturate the hardware encoder and decoder engines such that the initialization time overhead for one session gets hidden behind the transcoding time of other sessions. This can be achieved by running multiple parallel encode/decode sessions on the hardware (see Section [1:N Encode from YUV or RAW Data](#)). In such a case, the *aggregate* transcode performance with FFmpeg matches closely with the theoretically expected hardware performance.

7.1. Measuring Aggregate Performance

To measure GPU hardware accelerated aggregate performance, follow the steps below:

1. Run multiple simultaneous sessions (say 4 FFmpeg sessions) in parallel, each performing transcoding.
2. Ensure the inputs have large number of frames (more than 15 seconds of video is recommended) so that initialization time overhead can be ignored.
3. Measure the time required by each transcode.
4. Derive the aggregate performance in terms of frames per second (FPS).

7.2. Monitoring GPU and Codec Utilization with `nvidia-smi dmon`

Use `nvidia-smi dmon` to observe GPU utilization and PCIe traffic while FFmpeg runs. This helps verify that the pipeline is using the hardware efficiently and not bottlenecked by unnecessary data copies between GPU and host.

Run `nvidia-smi dmon` in a separate terminal (or in the background) while running your FFmpeg command. The `-s` option selects metrics; useful combinations include:

- ▶ `-s puc` (default): power (p), utilization (u), and clocks (c)
- ▶ `-s put`: power, utilization, and **PCIe Rx/Tx throughput (t)** — helpful to spot extra host-GPU traffic when frames are copied to or from host memory

```
nvidia-smi dmon -s put
```

Per-engine NVENC/NVDEC activity is available on GPM-capable GPUs via **GPU Performance Monitoring** with `--gpm-metrics` (e.g. `--gpm-metrics=30,166` for NVDEC 0 and NVENC 0). GPM codec metrics are supported on **Hopper and newer data-center/MIG-enabled GPUs** (e.g. H100, GB200); GeForce support is limited and many consumer boards will report 0/- for these fields even under load. See `nvidia-smi dmon -h` for the full list (NVDEC 0-7 = 30-37, NVENC 0-3 = 166-169). On all platforms, including GeForce, the standard **enc** and **dec** columns from `-s put` (or `-s u`) report legacy NVDEC/NVENC utilization — use these together with **rxpci/txpci** to compare pipelines when GPM is not available.

When transcoding is GPU-bound and efficient, you should see sustained GPU utilization (and enc/dec % when available); if frames are being copied to or from the host unnecessarily, PCIe throughput will be higher and end-to-end FPS lower.

7.3. Decoder Performance: Keep Frames on the GPU

When measuring decoder performance with a decode-to-null pipeline, `-hwaccel_output_format cuda` matters. Without it, the decoder outputs frames in host memory; FFmpeg then copies each frame from GPU to CPU before discarding it, which adds PCIe traffic and lowers the FPS reported by `-benchmark`. With `-hwaccel_output_format cuda`, decoded frames stay on the GPU and are discarded there, so the benchmark more closely reflects NVDEC throughput without the copy-to-host overhead.

Compare both commands (e.g. with `nvidia-smi dmon -s put` in a separate terminal to observe **dec** utilization and **rxpci/txpci**):

Without `-hwaccel_output_format cuda` — frames copied to host, lower FPS:

```
ffmpeg -benchmark -hwaccel cuda -i input.mp4 -f null -
```

With `-hwaccel_output_format cuda` — frames stay on GPU, higher FPS:

```
ffmpeg -benchmark -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -f  
→null -
```

The second command typically reports higher decode FPS and shows lower PCIe traffic in `nvidia-smi dmon`, since frames are not transferred to host memory.

7.4. Settings for Reduced Initialization Time

To prepare longer videos for streamed distribution, they are typically split into smaller chunks and each chunk is encoded separately. Such chunk-based encoding avoids error propagation, provides

clean boundaries for streaming bandwidth adaptation and helps parallelizing transcoding workloads on the servers. Transcoding smaller video chunks using GPU-hardware-accelerated transcoding, however, poses a challenge because the initialization time overhead of each FFmpeg process becomes significant.

To minimize the overhead when transcoding M input files into MN output files (i.e. when each of the M inputs is transcoded into N outputs), it is better to minimize the number of FFmpeg processes launched (see Section [1:N Encode from YUV or RAW Data](#) for example command lines).

Additionally, follow these tips to reduce the FFmpeg initialization time overhead:

- Set the following environment variables:

```
export CUDA_VISIBLE_DEVICES=0 # Use ID for the GPU device which you plan to
                               use for transcode
export CUDA_DEVICE_MAX_CONNECTIONS=2
```

- Use FFmpeg command lines such as those in Sections [1:N Transcode with Scaling](#) and [1:N Encode from YUV or RAW Data](#). These command lines share the CUDA context across multiple transcode sessions, thereby reducing the CUDA context initialization time overhead significantly.

Notices

Notice

This document is provided for information purposes only and shall not be regarded as a warranty of a certain functionality, condition, or quality of a product. NVIDIA Corporation (“NVIDIA”) makes no representations or warranties, expressed or implied, as to the accuracy or completeness of the information contained in this document and assumes no responsibility for any errors contained herein. NVIDIA shall have no liability for the consequences or use of such information or for any infringement of patents or other rights of third parties that may result from its use. This document is not a commitment to develop, release, or deliver any Material (defined below), code, or functionality.

NVIDIA reserves the right to make corrections, modifications, enhancements, improvements, and any other changes to this document, at any time without notice.

Customer should obtain the latest relevant information before placing orders and should verify that such information is current and complete.

NVIDIA products are sold subject to the NVIDIA standard terms and conditions of sale supplied at the time of order acknowledgment, unless otherwise agreed in an individual sales agreement signed by authorized representatives of NVIDIA and customer (“Terms of Sale”). NVIDIA hereby expressly objects to applying any customer general terms and conditions with regards to the purchase of the NVIDIA product referenced in this document. No contractual obligations are formed either directly or indirectly by this document.

NVIDIA products are not designed, authorized, or warranted to be suitable for use in medical, military, aircraft, space, or life support equipment, nor in applications where failure or malfunction of the NVIDIA product can reasonably be expected to result in personal injury, death, or property or environmental damage. NVIDIA accepts no liability for inclusion and/or use of NVIDIA products in such equipment or applications and therefore such inclusion and/or use is at customer’s own risk.

NVIDIA makes no representation or warranty that products based on this document will be suitable for any specified use. Testing of all parameters of each product is not necessarily performed by NVIDIA. It is customer’s sole responsibility to evaluate and determine the applicability of any information contained in this document, ensure the product is suitable and fit for the application planned by customer, and perform the necessary testing for the application in order to avoid a default of the application or the product. Weaknesses in customer’s product designs may affect the quality and reliability of the NVIDIA product and may result in additional or different conditions and/or requirements beyond those

contained in this document. NVIDIA accepts no liability related to any default, damage, costs, or problem which may be based on or attributable to: (i) the use of the NVIDIA product in any manner that is contrary to this document or (ii) customer product designs.

Trademarks

NVIDIA, the NVIDIA logo, and cuBLAS, CUDA, CUDA Toolkit, cuDNN, DALI, DIGITS, DGX, DGX-1, DGX-2, DGX Station, DLProf, GPU, Jetson, Kepler, Maxwell, NCCL, Nsight Compute, Nsight Systems, NVCaffe, NVIDIA Deep Learning SDK, NVIDIA Developer Program, NVIDIA GPU Cloud, NVLink, NVSHMEM, PerfWorks, Pascal, SDK Manager, Tegra, TensorRT, TensorRT Inference Server, Tesla, TF-TRT, Triton Inference Server, Turing, and Volta are trademarks and/or registered trademarks of NVIDIA Corporation in the United States and other countries. Other company and product names may be trademarks of the respective companies with which they are associated.

Copyright

©2022-2026, NVIDIA Corporation & affiliates. All rights reserved